BE-04 — Relatório de Teste da API do Feed

Tipo: Teste · **Responsável:** Back-end **Projeto:** Aplicativo baseado no YouTube **Autor:** Luiz **Data:** 21/10/2026

1. Objetivo

Testar o endpoint `GET /api/videos`, validando os campos retornados, o formato JSON e as respostas de erro.

Critério de aceitação: a API responde corretamente e os dados podem ser consumidos pelo Front-end.

2. Ambiente de teste

Item	Valor
Linguagem	Java (sem frameworks, usando `com.sun.net.httpserver.HttpServer`)
IDE	IntelliJ IDEA
Endereço da API	`http://localhost:8080/api/videos`
Ferramentas de teste	Navegador (Chrome), `curl` (Prompt de Comando), jsonlint.com
Sistema operacional	Windows

3. Resultado dos testes

#	Teste	Comando / Ação	Esperado	Resultado
1	Listagem de vídeos	`curl -i http://localhost:8080/api/videos`	Status 200	☑ Aprovado
2	Tipo de conteúdo	Cabeçalho `Content-Type`	`application/json; charset=utf-8`	☑ Aprovado
3	JSON válido	Colar a resposta no jsonlint.com	Valid JSON	☑ Aprovado
4	Campos de cada vídeo	Conferir a resposta	`id`, `titulo`, `urlThumbnail`, `nomeCanal`	☑ Aprovado
5	Quantidade de vídeos	Contar itens da lista	3	☑ Aprovado
6	URL inexistente	`curl -i http://localhost:8080/api/qua`	Status 404	☑ Aprovado

#	Teste	Comando / Ação	Esperado	Resultado
		`lquercoisa`		
7	Método não permitido	`curl -i -X POST http://localhost:8080/api/videos`	Status 405	☑ Aprovado (após correção)
8	CORS	Cabeçalho `Access-Control-Allow-Origin`	`*`	☑ Aprovado

4. Exemplo de resposta da API

```
[
  {
    "id": "v001",
    "titulo": "Construindo um castelo gigante no Minecraft",
    "urlThumbnail": "https://exemplo.com/thumbs/v001.jpg",
    "nomeCanal": "Rezendolas"
  },
  {
    "id": "v002",
    "titulo": "Aventura no Nether com shaders pesados",
    "urlThumbnail": "https://exemplo.com/thumbs/v002.jpg",
    "nomeCanal": "Leonam Edson"
  },
  {
    "id": "v003",
    "titulo": "Tutorial de Java para iniciantes",
    "urlThumbnail": "https://exemplo.com/thumbs/v003.jpg",
    "nomeCanal": "Canal Dev"
  }
]
```

5. Falha encontrada e correção

Problema: ao enviar `POST /api/videos`, a API respondia `200 OK` com a lista de vídeos. O correto é recusar métodos diferentes de GET.

Correção: foi adicionada uma verificação do método HTTP no início do endpoint. Se não for `GET`, a API responde `405 Method Not Allowed` com o cabeçalho `Allow: GET` e uma mensagem de erro em JSON.

Resultado após a correção: `POST /api/videos` passou a retornar `405 Method Not Allowed`.

7. Código final do servidor (`ServidorVideos.java`)

```
package Youtube_Archives;

import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
```

```
import java.nio.charset.StandardCharsets;
import java.util.List;

/**
 * BE-02 — API de listagem (Java puro, sem frameworks).
 * GET /api/videos -> retorna a lista de vídeos em JSON.
 * BE-04 — Ajuste: métodos diferentes de GET retornam 405.
 */
public class ServidorVideos {

    // Dados mocados (sem banco de dados)
    private static final List<Video> VIDEOS = List.of(
            new Video("v001", "Construindo um castelo gigante no Minecraft",
                    "https://exemplo.com/thumbs/v001.jpg", "Rezendolas"),
            new Video("v002", "Aventura no Nether com shaders pesados",
                    "https://exemplo.com/thumbs/v002.jpg", "Leonam Edson"),
            new Video("v003", "Tutorial de Java para iniciantes",
                    "https://exemplo.com/thumbs/v003.jpg", "Canal Dev")
    );

    public static void main(String[] args) throws IOException {
        HttpServer servidor = HttpServer.create(new InetSocketAddress(8080), 0);

        servidor.createContext("/api/videos", troca -> {

            // Só aceita GET
            if (!"GET".equalsIgnoreCase(troca.getRequestMethod())) {
                byte[] erro = "{\"erro\":\"Metodo nao
permitido\"}".getBytes(StandardCharsets.UTF_8);
                troca.getResponseHeaders().set("Content-Type", "application/json;
charset=utf-8");
                troca.getResponseHeaders().set("Allow", "GET");
                troca.sendResponseHeaders(405, erro.length);
                try (OutputStream saida = troca.getResponseBody()) {
                    saida.write(erro);
                }
                return;
            }

            byte[] resposta = montarJson().getBytes(StandardCharsets.UTF_8);

            troca.getResponseHeaders().set("Content-Type", "application/json;
charset=utf-8");
            // Permite que o front-end (outra porta/origem) acesse a API
            troca.getResponseHeaders().set("Access-Control-Allow-Origin", "*");
            troca.sendResponseHeaders(200, resposta.length);

            try (OutputStream saida = troca.getResponseBody()) {
                saida.write(resposta);
            }
        });

        servidor.start();
        System.out.println("Servidor rodando em http://localhost:8080/api/videos");
    }
```

```
    // Transforma a lista de vídeos em texto JSON
    private static String montarJson() {
        StringBuilder json = new StringBuilder("[");
        for (int i = 0; i < VIDEOS.size(); i++) {
            Video v = VIDEOS.get(i);
            json.append("{")
                    .append("\"id\":\"").append(escapar(v.getId())).append("\",")

.append("\"titulo\":\"").append(escapar(v.getTitulo())).append("\",")

.append("\"urlThumbnail\":\"").append(escapar(v.getUrlThumbnail())).append("\",")

.append("\"nomeCanal\":\"").append(escapar(v.getNomeCanal())).append("\"")
                    .append("}");
            if (i < VIDEOS.size() - 1) {
                json.append(",");
            }
        }
        return json.append("]").toString();
    }

    // Evita quebrar o JSON se o texto tiver aspas ou barras
    private static String escapar(String texto) {
        return texto.replace("\\", "\\\\").replace("\"", "\\\"");
    }
}
```

8. Conclusão

Todos os testes foram executados e aprovados. A API retorna status `200` com JSON válido, contendo os quatro campos esperados (`id`, `titulo`, `urlThumbnail`, `nomeCanal`), com `Content-Type` correto e CORS liberado. URLs inexistentes retornam `404` e métodos não permitidos retornam `405`.

Critério de aceitação atendido: a API responde corretamente e os dados podem ser consumidos pelo Front-end.